

Servidor Propio + Tiempo Real

Guía completa de migración y operación
Sistema General Olea Controls

React 19 + Vite

Express 5

Prisma 7

PostgreSQL

Socket.IO

Costo \$0

Documento técnico operativo — versión 1.0

Fecha: 17 de septiembre de 2026

Repositorio: `Sistema-General-Oleacontrols` (rama `main`)

Destino: PC local como servidor de producción con WebSockets

0. Índice y resumen ejecutivo

Qué resuelve este documento

Hoy tu sistema corre en **Vercel (serverless)**, con la base de datos en **Prisma Postgres (Accelerate)** y los archivos en **Cloudflare R2**. Ese modelo funciona, pero tiene dos límites duros:

- **No hay tiempo real.** Las funciones serverless de Vercel mueren al terminar la petición: no pueden mantener un WebSocket abierto ni una conexión SSE larga. Hoy tu app simula “vivo” con `setInterval` (polling) en `OTDetail.jsx`, `SupervisorOTs.jsx`, `TechnicianOTs.jsx` y `ApprovalsList.jsx`. Eso consume invocaciones, gasta datos móviles del técnico y siempre llega tarde.
- **Los planes gratuitos tienen techo.** Prisma Postgres gratis limita almacenamiento y operaciones; Vercel Hobby limita horas de función y ancho de banda, y su licencia restringe el uso comercial.

Pasar la aplicación a una **PC propia encendida 24/7** elimina las dos limitantes: un proceso Node siempre vivo sostiene WebSockets, y un Postgres local no cobra por GB ni por consulta. El costo de infraestructura baja a **\$0** (solo luz e internet que ya pagas).

Las tres rutas posibles

Ruta	Qué implica	Tiempo real	Costo / Riesgo
A — Híbrido	El frontend sigue en Vercel (CDN, HTTPS gratis). La API + WebSocket se mudan a tu PC. La BD sigue en Prisma Postgres.	Sí, vía la PC	\$0. Riesgo bajo, reversible en minutos. Sigues atado a los límites de Prisma.
B — Todo en casa (RECOMENDADA)	Frontend, API, WebSocket y PostgreSQL corren en tu PC. Vercel queda como respaldo de emergencia.	Sí, nativo	\$0. Riesgo medio: si se va la luz o el internet, el sistema se cae. Se mitiga con UPS + respaldo en Vercel.
C — Quedarte en Vercel	No mueves nada; mejoras el polling actual o contratas un servicio externo de realtime.	Parcial / de pago	\$0 hoy, pero el realtime de verdad requiere servicio pagado (Pusher, Ably) al crecer.

Recomendación

Haz

A primero

(un día de trabajo, sin tocar la base de datos) y cuando confirmes que la PC aguanta, pasa a

B

migrando Postgres. Así nunca te quedas sin sistema funcionando. Este documento cubre las dos fases en orden.

Contenido

1. Arquitectura actual vs. arquitectura destino
2. Requisitos: hardware, software y red
3. Fase 0 — Preparar la PC como servidor (Windows)
4. Fase 1 — Instalar el stack (Node, PostgreSQL, Git)

5. Fase 2 — Migrar la base de datos de Prisma Postgres a PostgreSQL local
6. Fase 3 — Servir la aplicación completa desde Express
7. Fase 4 — Publicar a internet con HTTPS, gratis (3 opciones)
8. Fase 5 — Implementar el tiempo real con Socket.IO (código incluido)
9. Fase 6 — Convertir el servidor en servicio de Windows (arranque automático)
10. Fase 7 — Respalos automáticos a Cloudflare R2
11. Fase 8 — Seguridad y endurecimiento
12. Fase 9 — Plan de contingencia y vuelta atrás
13. Anexos: comandos rápidos, checklist de puesta en marcha, solución de problemas

1. Arquitectura actual vs. destino

1.1 Cómo está hoy

```
NAVEGADOR / PWA del técnico
| https://tu-app.vercel.app
v
[ VERCEL ] index.html + assets (CDN)
| /api/* --> rewrite --> api/gateway.js (función serverless, 60 s máx.)
|                                     |
|                                     +--> api/_handlers/*.js (48 handlers)
|                                     +--> api/_lib/prisma.js
v                                     |
[ PRISMA POSTGRES ] prisma+postgres://...db.prisma.io (Accelerate, pooling administrado)
|
[ CLOUDFLARE R2 ] fotos, PDFs, firmas (S3 API + URLs prefiradas)
|
[ TELEGRAM ] notificaciones salientes (api/_lib/telegram.js)
```

Puntos clave que ya tienes resueltos y que **no cambian** al migrar:

- `api/_lib/prisma.js` ya detecta solo el tipo de conexión: si `DATABASE_URL` empieza con `prisma://` usa Accelerate; si empieza con `postgres://` abre un pool TCP con `@prisma/adapters-pg` y respeta `DB_POOL_MAX`. **Para migrar a Postgres local solo cambias esa variable.** Cero código.
- `server.js` ya monta los 48 handlers sobre Express 5 con el wrapper `adaptHandler`. Es literalmente el servidor de producción que necesitas; solo le faltan tres cosas: servir estáticos, WebSocket y correr como servicio.
- R2 y Telegram son servicios externos: siguen funcionando igual desde la PC.

1.2 Cómo queda (Ruta B, destino final)

```
NAVEGADOR / PWA del técnico
| https://olea.tudominio.com (HTTPS real, certificado válido)
v
[ CLOUDFLARE TUNNEL / TAILSCALE FUNNEL ] (sin abrir puertos del router)
| túnel saliente cifrado
v
[ TU PC - Windows, 24/7 ]
| node server.js (servicio de Windows, arranque automático)
| - express.static('dist') -> sirve el frontend compilado
| - /api/* -> los mismos 48 handlers
| - Socket.IO /socket.io -> TIEMPO REAL
| - pg Pool (DB_POOL_MAX=20)
v
[ POSTGRES 16 LOCAL ] localhost:5432 (sin límite de filas ni de storage)
v
pg_dump diario -> Cloudflare R2 (respaldo fuera de sitio)

[ VERCEL ] se mantiene desplegado como RESPALDO apuntando a Prisma Postgres
```

1.3 Qué se gana y qué se pierde

Aspecto	Vercel + Prisma (hoy)	PC propia (destino)
WebSockets / tiempo real	Imposible	Nativo, ilimitado
Límite de almacenamiento de BD	Plan gratuito acotado	El disco de la PC
Latencia de consulta	60–250 ms (red hasta db.prisma.io)	1–5 ms (localhost)
Arranque en frío	Sí, 300–900 ms en la primera petición	No existe
Tareas programadas (cron)	Limitadas en plan gratis	Task Scheduler, sin límite
Subidas grandes	Tope de 20 MB por función	Lo que tú definas
Disponibilidad	99.99 % (SLA de Vercel)	Depende de tu luz e internet
Mantenimiento	Cero	Tuyo: parches, respaldos, monitoreo
Costo mensual	\$0 (con límites)	\$0 + ~\$60–120 MXN de luz

Sé honesto con el riesgo

Una PC doméstica no tiene SLA. Un apagón de CFE, un corte de internet o una actualización de Windows que reinicie el equipo dejan el sistema fuera hasta que alguien lo atienda. Por eso la Fase 9 (contingencia) **no es opcional**

: mantener el deploy de Vercel vivo cuesta \$0 y te salva el día que la PC falle.

2. Requisitos

2.1 Hardware mínimo y recomendado

Componente	Mínimo funcional	Recomendado (50 usuarios)	Por qué
CPU	2 núcleos / Core i3 8.ª gen	4 núcleos / i5 o Ryzen 5	Node es monohilo, pero Postgres y el build sí usan varios núcleos
RAM	8 GB	16 GB	Postgres ~2 GB + Node ~500 MB + Windows ~4 GB
Disco	SSD 256 GB	SSD NVMe 500 GB	Postgres sobre HDD es 10–20× más lento. SSD obligatorio.
Red	20 Mbps de subida	50 Mbps de subida, cable Ethernet	La subida es la que sirve a tus usuarios, no la bajada. Nunca por Wi-Fi.
Energía	—	UPS/No-break 600 VA	Un corte de luz con Postgres escribiendo puede corromper datos

Sobre la PC que uses

No necesita ser una máquina nueva. Un equipo de oficina con i5 de 8.ª generación, 16 GB y SSD sostiene sin problema los 50 usuarios que ya tienes proyectados. Lo que

sí

importa: que esté en un lugar ventilado, conectada por cable Ethernet, con UPS, y que

nadie la use como PC de escritorio

(un Excel pesado o un antivirus escaneando compiten contra Postgres).

2.2 Software a instalar

Software	Versión	Para qué	Costo
Node.js LTS	20.x o 22.x	Ejecutar <code>server.js</code> . Tu <code>package.json</code> declara <code>engines: node 20.x</code>	Gratis
PostgreSQL	16 o 17	Base de datos local (Fase 2, ruta B)	Gratis
Git for Windows	Última	Traer actualizaciones del repo con <code>git pull</code>	Gratis
PM2 + pm2-windows-service	Última	Mantener Node vivo y arrancarlo con Windows	Gratis
cloudflared o Tailscale	Última	Publicar a internet con HTTPS sin abrir puertos	Gratis
pgAdmin 4	Última	Administrar la BD con interfaz gráfica	Gratis

2.3 Cuentas que ya tienes y siguen sirviendo

- **Cloudflare** — ya la usas para R2. La misma cuenta te da Tunnel gratis y DNS gratis.
- **Vercel** — se queda como respaldo y como CDN del frontend en la Ruta A.
- **Prisma Data Platform** — se queda como respaldo de BD; de ahí sacarás el volcado inicial.

- **GitHub** — origen del código que la PC va a clonar.

3. Fase 0 — Preparar la PC como servidor

Tiempo estimado: 45 minutos. Todo se hace en la PC que será el servidor, con una cuenta de administrador.

3.1 Que la PC nunca se duerma

Un servidor que entra en suspensión deja de responder. Abre PowerShell **como administrador** y ejecuta:

```
# Nunca suspender, nunca apagar disco ni pantalla (estando conectada a corriente)
powercfg /change standby-timeout-ac 0
powercfg /change hibernate-timeout-ac 0
powercfg /change disk-timeout-ac 0
powercfg /change monitor-timeout-ac 10

# Deshabilitar hibernación por completo (libera varios GB de disco)
powercfg /hibernate off

# Si es laptop: que al cerrar la tapa NO haga nada
powercfg /setacvalueindex SCHEME_CURRENT 4f971e89-eebd-4455-a8de-9e59040e7347 5ca83367-6e45-459f-a27b-476b1d01c936 0
powercfg /setactive SCHEME_CURRENT

# Que encienda sola tras un corte de luz (esto se activa en la BIOS/UEFI:
# busca "Restore on AC Power Loss" o "After Power Failure" -> Power On)
```

3.2 Arranque automático de sesión (opcional pero muy útil)

Si usas PM2 como servicio (Fase 6) no hace falta. Si prefieres simplicidad, configura inicio de sesión automático con `netplwiz` y quita la casilla “Los usuarios deben escribir su nombre y contraseña”.

3.3 Actualizaciones de Windows controladas

Lo peor que le puede pasar a tu servidor es un reinicio sorpresa a media jornada. Configura las horas activas y pausa los reinicios automáticos:

```
# Horas activas 07:00 a 22:00 — Windows no reinicia dentro de esa ventana
reg add "HKLM\SOFTWARE\Microsoft\WindowsUpdate\UX\Settings" /v ActiveHoursStart /t REG_DWORD /d 7 /f
reg add "HKLM\SOFTWARE\Microsoft\WindowsUpdate\UX\Settings" /v ActiveHoursEnd /t REG_DWORD /d 22 /f
```

3.4 IP local fija

La PC debe tener siempre la misma IP en la red. Lo más limpio es reservarla en el router por dirección MAC (busca “DHCP Reservation” en la configuración del módem). Alternativa desde Windows:

```
# Ver el nombre y la configuración actual del adaptador
Get-NetIPConfiguration

# Fijar IP (ajusta nombre de interfaz, IP, prefijo y gateway a tu red)
New-NetIPAddress -InterfaceAlias "Ethernet" -IPAddress 192.168.1.50 -PrefixLength 24 -DefaultGateway 192.168.1.1
Set-DnsClientServerAddress -InterfaceAlias "Ethernet" -ServerAddresses 1.1.1.1,8.8.8.8
```

3.5 Carpetas de trabajo

No pongas el proyecto dentro de OneDrive

Hoy el repo vive en `C:\Users\riveh\OneDrive\Escritorio\n2\...`. OneDrive sincroniza `node_modules`, bloquea archivos mientras los sube y puede corromper `.git`. Para el servidor, clona el repositorio **fuera**

de OneDrive:

```
New-Item -ItemType Directory -Force C:\olea
New-Item -ItemType Directory -Force C:\olea\backups
New-Item -ItemType Directory -Force C:\olea\logs
cd C:\olea
git clone https://github.com/OleaControls/Sistema-General-Oleacontrols.git app
# Ajusta la URL real de tu remoto: git remote -v en tu equipo actual la muestra
```

4. Fase 1 — Instalar el stack

Tiempo estimado: 30 minutos. Todo con `winget`, que ya viene en Windows 11.

4.1 Instalación en una sola pasada

```
# PowerShell como ADMINISTRADOR
winget install OpenJS.NodeJS.LTS
winget install Git.Git
winget install PostgreSQL.PostgreSQL.16
winget install Cloudflare.cloudflare

# Cierra y vuelve a abrir PowerShell para que tome el PATH, luego verifica:
node -v      # debe decir v20.x o v22.x
npm -v
git --version
psql --version
cloudflared --version
```

Durante la instalación de PostgreSQL

El instalador pide una contraseña para el usuario `postgres`.

Anótala en tu gestor de contraseñas

: sin ella no puedes administrar la base. Deja el puerto en `5432` y el locale en `Spanish, Mexico` o `C`.

4.2 Instalar dependencias del proyecto

```
cd C:\olea\app
npm ci      # instala exactamente lo del package-lock.json; el postinstall corre prisma generate
```

Si `npm ci` falla por el `postinstall` (necesita `DATABASE_URL` para generar el cliente), crea primero el archivo `.env` de la Fase 2 y vuelve a intentar.

4.3 Archivo `.env` del servidor

Copia el `.env` que ya tienes y ajústalo. Estructura final para la PC:

```
# === C:\olea\app\.env ===

# RUTA A (híbrido): sigues en Prisma Postgres, no cambias nada aquí
# DATABASE_URL="prisma+postgres://accelerate.prisma-data.net/?api_key=..."

# RUTA B (todo local): apunta a tu Postgres. prisma.js detecta el prefijo
# postgresql:// y abre pool TCP automáticamente, sin tocar código.
DATABASE_URL="postgresql://olea_app:CONTRASENA_FUERTE@localhost:5432/olea?schema=public"
DIRECT_URL="postgresql://olea_app:CONTRASENA_FUERTE@localhost:5432/olea?schema=public"

# Pool: en un servidor siempre encendido conviene subirlo (prisma.js lee esta var)
DB_POOL_MAX=20

# Los mismos que ya usas – R2 no cambia
R2_BUCKET_NAME="..."
R2_ACCESS_KEY="..."
R2_SECRET_KEY="..."
R2_ENDPOINT="https://XXXXX.r2.cloudflarestorage.com"
R2_REGION="auto"
R2_PUBLIC_URL="https://..."

JWT_SECRET="el_mismo_que_ya_tienes_para_no_invalidar_sesiones"
TELEGRAM_BOT_TOKEN="..."

# Nuevas para el servidor propio
PORT=3001
NODE_ENV=production
APP_ORIGIN="https://olea.tudominio.com"
```

Ojo con SSL en Postgres local

`api/_lib/prisma.js` crea el pool con `ssl: { rejectUnauthorized: false }`. Un PostgreSQL local recién instalado

no tiene SSL habilitado

y rechazará la conexión. Tienes dos salidas, ambas válidas — la 2 es la recomendada por ser un cambio de una línea en el código:

- 1) Habilitar SSL en `postgresql.conf` con certificado autofirmado.
- 2) Hacer el SSL condicional (ver Fase 2.5).

5. Fase 2 — Migrar la base de datos a PostgreSQL local

Solo aplica a la Ruta B. Tiempo estimado: 1–2 horas. Hazlo en fin de semana o fuera de horario.

5.1 Crear la base y el usuario de la aplicación

Nunca uses el superusuario `postgres` para la app. Crea un usuario dedicado:

```
psql -U postgres

-- dentro de psql:
CREATE DATABASE olea WITH ENCODING 'UTF8';
CREATE USER olea_app WITH PASSWORD 'CONTRASENA_FUERTE';
GRANT ALL PRIVILEGES ON DATABASE olea TO olea_app;
\c olea
GRANT ALL ON SCHEMA public TO olea_app;
ALTER SCHEMA public OWNER TO olea_app;
\q
```

5.2 Crear el esquema (63 modelos) con Prisma

Tu `prisma/schema.prisma` tiene 1,657 líneas y 63 modelos. No los crees a mano: deja que Prisma los genere en la base vacía.

```
cd C:\olea\app

# Si ya tienes carpeta prisma/migrations en el repo:
npx prisma migrate deploy

# Si NO manejas migraciones versionadas (usaste db push históricamente):
npx prisma db push

# Verifica que las tablas existan
psql -U olea_app -d olea -c "\dt"
```

5.3 Traer los datos desde Prisma Postgres

Aquí está el único punto delicado de toda la migración. Tu `DATABASE_URL` actual es `prisma+postgres://...db.prisma.io`, que es un **protocolo HTTP propietario**: `pg_dump` no puede conectarse a esa cadena. Tienes dos caminos.

Camino 1 — Cadena de conexión directa (el mejor, si tu plan la ofrece)

En la consola de Prisma Data Platform, dentro de tu proyecto, busca la sección de conexión y localiza la opción de **conexión TCP directa** (aparece como “Direct connection”, “TCP connection string” o similar, con formato `postgres://usuario:clave@host:5432/db`). Con esa cadena:

```
# 1) Volcado completo desde la nube (solo datos, el esquema ya lo creó Prisma)
pg_dump --data-only --no-owner --no-privileges --disable-triggers `
  --dbname="postgres://USUARIO:CLAVE@HOST:5432/BASE?sslmode=require" `
  --file=C:\olea\backups\olea-datos.sql

# 2) Restaurar en el Postgres local
psql -U olea_app -d olea -f C:\olea\backups\olea-datos.sql

# 3) Reajustar las secuencias de los IDs autoincrementales
psql -U olea_app -d olea -c "SELECT setval(pg_get_serial_sequence(quote_ident(c.relname), a.attname),
COALESCE((SELECT MAX(x.id) FROM pg_catalog.pg_class y WHERE false), 1)) FROM pg_class c JOIN pg_attribute a
ON a.attrelid=c.oid WHERE false;"
```

Nota: el paso 3 depende de tus tipos de ID. Si usas `cuid()` / `uuid()` en el schema (lo habitual en Prisma) **no hay secuencias que ajustar y puedes omitirlo**. Solo aplica a campos `Int @id @default(autoincrement())`; en ese caso corre, por cada tabla afectada:

```
SELECT setval('Tabla_id_seq', (SELECT MAX(id) FROM "Tabla"));
```

Camino 2 — Copia tabla por tabla con Prisma (siempre funciona)

Si no tienes cadena directa, copia los datos con un script que usa dos clientes Prisma a la vez: uno leyendo de la nube y otro escribiendo en local. Crea `scripts/migrar-datos.js`:

```
// scripts/migrar-datos.js – copia datos de Prisma Postgres a Postgres local
import { PrismaClient } from '@prisma/client'
import { PrismaPg } from '@prisma/adapters-pg'
import { withAccelerate } from '@prisma/extension-accelerate'
import pg from 'pg'
import dotenv from 'dotenv'
dotenv.config()

const origen = new PrismaClient({ accelerateUrl: process.env.CLOUD_DATABASE_URL })
  .extends(withAccelerate())

const destino = new PrismaClient({
  adapter: new PrismaPg(new pg.Pool({ connectionString: process.env.LOCAL_DATABASE_URL })),
})

// ORDEN IMPORTANTE: primero las tablas sin llaves foráneas (padres),
// luego las que dependen de ellas. Ajusta según tu schema.prisma.
const ORDEN = [
  'user', 'employee', 'category', 'client', 'supplier', 'product',
  'workOrder', 'project', 'quote', 'purchaseOrder',
  /* ...continúa con el resto de tus 63 modelos... */
]

const LOTE = 500

for (const modelo of ORDEN) {
  if (!origen[modelo]) { console.warn(`omitido (no existe): ${modelo}`); continue }
  const total = await origen[modelo].count()
  console.log(`\n${modelo}: ${total} registros`)
  for (let i = 0; i < total; i += LOTE) {
    const filas = await origen[modelo].findMany({ skip: i, take: LOTE })
    await destino[modelo].createMany({ data: filas, skipDuplicates: true })
    process.stdout.write(` ${Math.min(i + LOTE, total)}/${total}\r`)
  }
}

await origen.$disconnect(); await destino.$disconnect()
console.log('\nMigración terminada.')
```

```
# Ejecutar (define las dos URLs temporalmente en el .env)
# CLOUD_DATABASE_URL="prisma+postgres://..."
# LOCAL_DATABASE_URL="postgresql://olea_app:...@localhost:5432/olea"
node scripts/migrar-datos.js
```

Cómo obtener el orden correcto de tablas

Sácalo del propio schema: los modelos que

no

tienen ningún campo de relación con `@relation(fields: [...])` van primero; los que sí, después de aquellos a los que apuntan. Si una inserción falla con `Foreign key constraint violated`, mueve ese modelo más abajo en el arreglo y vuelve a correr (por `skipDuplicates: true` el script es re-ejecutable sin duplicar).

5.4 Verificar la migración

```
# Compara conteos de las tablas críticas contra la nube antes de dar el cambio por bueno
psql -U olea_app -d olea -c "SELECT relname AS tabla, n_live_tup AS filas FROM pg_stat_user_tables ORDER BY
n_live_tup DESC LIMIT 25;"
```

Revisa manualmente: número de empleados, de OTs, de cotizaciones, de proyectos y de órdenes de compra. Si cuadran, la migración está bien.

5.5 Ajuste del SSL en prisma.js

Un cambio de una línea en `api/_lib/prisma.js` para que el pool no exija SSL cuando la base es local:

```
// ANTES
ssl: { rejectUnauthorized: false },

// DESPUÉS – sin SSL contra localhost, con SSL flexible contra cualquier host remoto
ssl: /@(localhost|127\.0\.0\.1)/.test(DB_URL) ? false : { rejectUnauthorized: false },
```

Con eso el mismo archivo sigue sirviendo para Vercel (Prisma Postgres, con SSL) y para la PC (Postgres local, sin SSL). No rompes el respaldo.

6. Fase 3 — Servir toda la aplicación desde Express

Tiempo estimado: 30 minutos. Convierte `server.js` en un servidor de producción completo.

6.1 El problema a resolver

Hoy `server.js` solo sirve `/api/*`; el frontend lo entrega Vite en desarrollo y Vercel en producción. En la PC, Express tiene que hacer las dos cosas: entregar el `dist/` compilado y responder la API. Además, como es una SPA con React Router, cualquier ruta desconocida debe devolver `index.html`.

6.2 Cambios en server.js

Agrega esto **al inicio**, junto a los demás imports:

```
import path from 'path';
import { fileURLToPath } from 'url';
const __dirname = path.dirname(fileURLToPath(import.meta.url));
```

Reemplaza `app.use(cors())` por una configuración restrictiva (Fase 8 explica por qué):

```
const ORIGENES = [
  process.env.APP_ORIGIN,           // https://olea.tudominio.com
  'https://tu-app.vercel.app',      // respaldo en Vercel
  'http://localhost:5173',          // Vite en desarrollo
].filter(Boolean);

app.use(cors({
  origin: (origin, cb) => cb(null, !origin || ORIGENES.includes(origin)),
  credentials: true,
}));
```

Y agrega esto **al final**, justo antes de `app.listen(...)` (el orden importa: los estáticos van después de todas las rutas `/api`):

```
// ----- Frontend compilado (SPA) -----
const DIST = path.join(__dirname, 'dist');

app.use(express.static(DIST, {
  index: false,
  setHeaders: (res, ruta) => {
    if (ruta.includes(`${path.sep}assets${path.sep}`)) {
      // Los chunks de Vite llevan hash: son inmutables
      res.setHeader('Cache-Control', 'public, max-age=31536000, immutable');
    } else if (ruta.endsWith('.sw.js') || ruta.endsWith('index.html')) {
      // El service worker y el HTML nunca se cachean (misma regla que vercel.json)
      res.setHeader('Cache-Control', 'no-cache, no-store, must-revalidate');
    }
  },
}));

// Cualquier ruta que no sea /api la resuelve React Router
app.get(/^(?!\/api\/).*/, (req, res) => {
  res.setHeader('Cache-Control', 'no-cache, no-store, must-revalidate');
  res.sendFile(path.join(DIST, 'index.html'));
});
```

Express 5 y los comodines

En Express 5

ya no existe

`app.get('*', ...)`: el parser de rutas cambió y esa línea revienta con `PathError`. Por eso arriba se usa una

expresión regular

`(/^(?!\/api\/).*/)`, que además excluye `/api` de una vez. Es el error número uno al montar SPAs sobre Express 5.

6.3 Compilar y probar

```
cd C:\olea\app
npm run build          # prisma generate + vite build -> genera dist/
node server.js

# En otra ventana, prueba que responda todo:
curl http://localhost:3001/api/config
curl -I http://localhost:3001/      # debe devolver 200 y text/html
curl -I http://localhost:3001/ots  # ruta de React: también 200 + index.html
```

Abre `http://192.168.1.50:3001` desde otra computadora de la red local. Si carga e inicias sesión, el servidor ya funciona; lo único que falta es sacarlo a internet.

6.4 Script de actualización

Crea `C:\olea\actualizar.ps1` para desplegar cambios sin recordar comandos:

```
# C:\olea\actualizar.ps1
Set-Location C:\olea\app
Write-Host "Respaldando base de datos..." -ForegroundColor Cyan
$fecha = Get-Date -Format "yyyyMMdd-HH:mm:ss"
pg_dump -U olea_app -d olea -Fc -f "C:\olea\backups\pre-deploy-$fecha.dump"

Write-Host "Trayendo cambios..." -ForegroundColor Cyan
git pull
npm ci
npx prisma migrate deploy
npm run build

Write-Host "Reiniciando servicio..." -ForegroundColor Cyan
pm2 restart olea-server
pm2 logs olea-server --lines 30
```

7. Fase 4 — Publicar a internet con HTTPS, gratis

Tiempo estimado: 30–60 minutos. Elige UNA de las tres opciones.

Por qué necesitas HTTPS sí o sí

Tu app es una

PWA

(vite-plugin-pwa). Los service workers

solo funcionan bajo HTTPS

(o localhost). Además la geolocalización de tech-locations y la cámara para fotos de OT están bloqueadas por el navegador en sitios HTTP. Sin HTTPS, la mitad del sistema deja de servir en campo.

7.1 Comparativa de las tres opciones

Opción	Costo real	URL	Ventajas	Desventajas
1. Cloudflare Tunnel RECOMENDADA	\$0 el túnel ~\$200 MXN/año el dominio	olea.tudominio.com	Ancho de banda ilimitado, WAF, protección DDoS, no abres puertos, HTTPS automático, Cloudflare Access para proteger rutas de admin	Requiere un dominio propio en Cloudflare
2. Tailscale Funnel	\$0 total	pc-olea.tu-tailnet.ts.net	100 % gratis, sin dominio, HTTPS automático, URL estable, 3 min de configuración	URL fea; ancho de banda con límites de uso justo; menos apto para cara al cliente
3. Puertos + DuckDNS + Caddy	\$0 total	oleacontrols.duckdns.org	Sin intermediarios, máximo rendimiento, control total	No funciona con CGNAT (la mayoría de ISP mexicanos residenciales). Expones tu IP y tu router

7.2 Opción 1 — Cloudflare Tunnel (paso a paso)

```
# 1) Autenticar cloudflared con tu cuenta (abre el navegador)
cloudflared tunnel login

# 2) Crear el túnel – guarda el ID que imprime
cloudflared tunnel create olea-server

# 3) Apuntar el subdominio al túnel (crea el registro DNS solo)
cloudflared tunnel route dns olea-server olea.tudominio.com
```

Crea el archivo de configuración en C:\Users\TU_USUARIO\.cloudflared\config.yml :

```
tunnel: olea-server
credentials-file: C:\Users\TU_USUARIO\.cloudflared\ID-DEL-TUNEL.json

ingress:
- hostname: olea.tudominio.com
  service: http://localhost:3001
  originRequest:
    noTLSVerify: true
    connectTimeout: 30s
    # Clave para WebSockets: sin esto Socket.IO se desconecta cada 100 s
    keepAliveTimeout: 90s
    httpHostHeader: olea.tudominio.com
- service: http_status:404
```

```
# 4) Probar en primer plano
cloudflared tunnel run olea-server
```

```
# 5) Si todo carga bien, instalarlo como servicio de Windows (arranque automático)
cloudflared service install
```

WebSockets en Cloudflare

Cloudflare soporta WebSockets en el

plan gratuito

, sin configuración extra. Solo verifica en el panel: *Network* → *WebSockets* = *On*. Socket.IO funcionará a través del túnel sin cambios.

7.3 Opción 2 — Tailscale Funnel (100 % gratis, sin dominio)

```
winget install tailscale.tailscale
tailscale up

# Exponer el puerto 3001 a internet con HTTPS
tailscale funnel --bg 3001

# Ver la URL pública asignada
tailscale funnel status
```

Te queda una URL permanente del tipo <https://pc-olea.tu-tailnet.ts.net>, con certificado válido, sin abrir un solo puerto. Es la opción más rápida para **probar la Ruta A hoy mismo** y después migrar a Cloudflare Tunnel cuando compres el dominio.

7.4 Opción 3 — Puerto abierto + DuckDNS + Caddy

```
# Antes de nada: comprueba si tienes IP pública real o CGNAT.
# Compara la IP WAN que muestra tu router contra la de ifconfig.me:
curl ifconfig.me
# Si NO coinciden, estás detrás de CGNAT y esta opción NO te sirve.
```

Si sí coinciden: registra un subdominio gratis en duckdns.org, abre los puertos 80 y 443 hacia la IP local de la PC en tu router, e instala Caddy como proxy inverso con HTTPS automático ([Caddyfile](#)):

```
oleacontrols.duckdns.org {  
    reverse_proxy localhost:3001  
}
```

```
winget install CaddyServer.Caddy  
caddy run --config C:\olea\Caddyfile
```

7.5 Apuntar el frontend al nuevo servidor (solo Ruta A)

Si dejas el frontend en Vercel y la API en la PC, hay que decirle a Vercel a dónde mandar `/api/*`. Edita `vercel.json`:

```
"rewrites": [  
  { "source": "/api/(.*)", "destination": "https://olea.tudominio.com/api/$1" },  
  { "source": "/(.*)", "destination": "/index.html" }  
]
```

Y asegúrate de que `APP_ORIGIN` y la lista `ORIGENES` del CORS (Fase 3) incluyan el dominio de Vercel. Como `src/lib/api.js` usa rutas relativas (`/api/...`), **no hay que tocar ni una línea del frontend**.

8. Fase 5 — Tiempo real con Socket.IO

Tiempo estimado: 2–3 horas. Esta es la razón de fondo de toda la migración.

8.1 Qué vas a lograr

Escenario	Hoy (polling)	Con Socket.IO
Técnico cierra una OT	El supervisor la ve hasta 30–60 s después	Aparece al instante en su pantalla
GPS de técnicos en el mapa	Se refresca por intervalo	El marcador se mueve en vivo
Aprobación de gastos	<code>setInterval</code> en <code>ApprovalsList</code>	Notificación push al aprobador
Consumo de datos del técnico	Una petición cada X segundos, todo el día	Una conexión abierta, tráfico solo cuando hay cambios
Carga sobre la BD	N usuarios × consultas por minuto	Casi cero en reposo

8.2 Instalación

```
cd C:\olea\app
npm install socket.io socket.io-client
```

8.3 Backend — crear `api/_lib/realtime.js`

Un módulo único que cualquier handler puede importar para emitir eventos. Usa el mismo JWT que ya emite `api/_handlers/login.js`, así que no hay un segundo sistema de autenticación.

```

// api/_lib/realtime.js
import { Server } from 'socket.io';
import jwt from 'jsonwebtoken';

let io = null;

export function initRealtime(httpServer, origenes) {
  io = new Server(httpServer, {
    cors: { origin: origenes, credentials: true },
    // Primero long-polling y luego sube a WebSocket: sobrevive a redes móviles malas
    transports: ['polling', 'websocket'],
    pingInterval: 25000,
    pingTimeout: 20000,
  });

  // Autenticación: mismo token que usa src/lib/api.js (localStorage 'olea_token')
  io.use((socket, next) => {
    const token = socket.handshake.auth?.token;
    if (!token) return next(new Error('sin token'));
    try {
      const payload = jwt.verify(token, process.env.JWT_SECRET);
      socket.data.user = payload;
      next();
    } catch {
      next(new Error('token inválido'));
    }
  });

  io.on('connection', (socket) => {
    const u = socket.data.user;
    // Salas: una por usuario y una por rol. Así se emite con precisión.
    socket.join(`user:${u.id}`);
    if (u.role) socket.join(`role:${u.role}`);
    console.log(`[RT] conectado ${u.email ?? u.id} (${u.role}) - total ${io.engine.clientsCount}`);

    // El cliente pide seguir una OT concreta al abrir su detalle
    socket.on('ot:seguir', (otId) => socket.join(`ot:${otId}`));
    socket.on('ot:dejar', (otId) => socket.leave(`ot:${otId}`));
    socket.on('proyecto:seguir', (id) => socket.join(`proyecto:${id}`));

    socket.on('disconnect', (motivo) => console.log(`[RT] desconectado ${u.id}: ${motivo}`));
  });

  return io;
}

/** Emite un evento a una sala. Seguro de llamar aunque el realtime esté apagado
 * (por ejemplo, cuando el mismo handler corre en Vercel como respaldo). */
export function emitir(sala, evento, datos) {
  if (!io) return false;
  io.to(sala).emit(evento, datos);
  return true;
}

export function emitirATodos(evento, datos) {
  if (!io) return false;
  io.emit(evento, datos);
  return true;
}

export const conectados = () => (io ? io.engine.clientsCount : 0);

```

8.4 Enganchar Socket.IO en server.js

Hay que reemplazar `app.listen()` por un servidor HTTP explícito, porque Socket.IO necesita adjuntarse a él:

```
// arriba, con los imports
import http from 'http';
import { initRealtime, conectados } from '../api/_lib/realtime.js';

// abajo, EN LUGAR DE app.listen(PORT, ...)
const server = http.createServer(app);
initRealtime(server, ORIGENES);

// Endpoint de salud: sirve para monitoreo y para saber cuántos están conectados
app.get('/api/health', (req, res) => {
  res.json({ ok: true, uptime: process.uptime(), realtime: conectados() });
});

server.listen(PORT, '0.0.0.0', () => {
  console.log(`🚀 Servidor en http://localhost:${PORT} - realtime activo`);
});
```

8.5 Emitir eventos desde los handlers

Ejemplo real sobre `api/_handlers/ots.js`. Después de guardar en la base, avisas a quien corresponde:

```
// api/_handlers/ots.js
import { emitir } from '../_lib/realtime.js';

// ... dentro del PUT que actualiza una OT, después del prisma.workOrder.update(...)
const otActualizada = await prisma.workOrder.update({ where: { id }, data });

// A quien tenga abierto el detalle de esa OT
emitir(`ot:${id}`, 'ot:actualizada', otActualizada);

// A los supervisores, para su tablero
emitir('role:SUPERVISOR', 'ot:cambio', { id, status: otActualizada.status });

// Al técnico asignado, aunque tenga otra pantalla abierta
if (otActualizada.technicianId) {
  emitir(`user:${otActualizada.technicianId}`, 'notificacion', {
    tipo: 'ot',
    titulo: `OT ${otActualizada.code} actualizada`,
    otId: id,
  });
}
```

Regla de oro

`emitir()` devuelve `false` y no hace nada si el realtime no está inicializado. Eso significa que **puedes agregar estas llamadas y seguir desplegando en Vercel sin romper nada**: allá simplemente no emiten. Un solo código base para los dos entornos.

8.6 Frontend — hook src/hooks/useRealtime.js

```
// src/hooks/useRealtime.js
import { useEffect, useRef } from 'react';
import { io } from 'socket.io-client';

let socketGlobal = null;

/** Conexión única y compartida por toda la app. */
export function getSocket() {
  if (socketGlobal) return socketGlobal;
  const token = localStorage.getItem('olea_token');
  if (!token) return null;

  socketGlobal = io(import.meta.env.VITE_RT_URL || window.location.origin, {
    auth: { token },
    transports: ['polling', 'websocket'],
    reconnection: true,
    reconnectionDelay: 1000,
    reconnectionDelayMax: 10000,
  });

  socketGlobal.on('connect_error', (e) => console.warn('[RT] error:', e.message));
  return socketGlobal;
}

export function cerrarSocket() {
  socketGlobal?.disconnect();
  socketGlobal = null;
}

/** Suscribe un manejador a un evento mientras el componente esté montado. */
export function useRealtime(evento, manejador, deps = []) {
  const ref = useRef(manejador);
  ref.current = manejador;

  useEffect(() => {
    const s = getSocket();
    if (!s) return;
    const fn = (...args) => ref.current(...args);
    s.on(evento, fn);
    return () => s.off(evento, fn);
  }, [evento, ...deps]);
}

/** Entra y sale de una sala (detalle de OT, de proyecto, etc.). */
export function useSala(eventoEntrar, eventoSalir, id) {
  useEffect(() => {
    const s = getSocket();
    if (!s || !id) return;
    s.emit(eventoEntrar, id);
    return () => s.emit(eventoSalir, id);
  }, [eventoEntrar, eventoSalir, id]);
}
```

8.7 Uso en las vistas — y retiro del polling

En `src/modules/ots/OTDetail.jsx`, sustituye el `setInterval` por:

```
import { useRealtime, useSala } from '@/hooks/useRealtime';

// dentro del componente, con otId ya resuelto
useSala('ot:seguir', 'ot:dejar', otId);

useRealtime('ot:actualizada', (ot) => {
  if (ot.id === otId) setOt(ot);          // refresco instantáneo
}, [otId]);

// ELIMINA el bloque: useEffect(() => { const t = setInterval(cargarOT, 30000); ... })
```

En `SupervisorOTs.jsx`, para el tablero:

```
useRealtime('ot:cambio', ({ id, status }) => {
  setOts((prev) => prev.map((o) => (o.id === id ? { ...o, status } : o)));
});
```

Y en el logout (donde hoy borras `olea_token`), llama `cerrarSocket()` para no dejar la conexión abierta con un token viejo.

Qué polling conservar

Deja un `setInterval` de respaldo

muy largo

(5 minutos) en las vistas críticas, activo solo si el socket está desconectado. El técnico en campo pierde señal constantemente; el realtime es la vía rápida, el polling lento es la red de seguridad:

```
if (!getSocket()?.connected) recargar();
```

9. Fase 6 — Servicio de Windows con PM2

Tiempo estimado: 20 minutos. Sin esto, el sistema se cae en cuanto cierres la ventana de PowerShell.

9.1 Instalación y arranque

```
npm install -g pm2
npm install -g pm2-windows-startup
pm2-startup install
```

Crea `C:\olea\app\ecosystem.config.cjs`:

```
module.exports = {
  apps: [{
    name: 'olea-server',
    script: 'server.js',
    cwd: 'C:\\olea\\app',
    instances: 1,                // 1 SOLA instancia: con Socket.IO el cluster
                                // requiere un adaptador Redis. No lo subas.
    exec_mode: 'fork',
    autorestart: true,
    max_memory_restart: '1G',    // se reinicia solo si hay fuga de memoria
    watch: false,
    env: { NODE_ENV: 'production' },
    error_file: 'C:\\olea\\logs\\error.log',
    out_file: 'C:\\olea\\logs\\salida.log',
    time: true,                  // marca de tiempo en cada línea de log
  }],
};
```

```
cd C:\olea\app
pm2 start ecosystem.config.cjs
pm2 save                # graba la lista para que reviva tras el reinicio

# Comprobar que sobrevive a un reinicio de Windows:
Restart-Computer
# ... al volver:
pm2 list
pm2 logs olea-server
```

9.2 Comandos de operación diaria

Comando	Qué hace
<code>pm2 list</code>	Estado, memoria, CPU, reinicios de cada proceso
<code>pm2 logs olea-server</code>	Logs en vivo
<code>pm2 logs olea-server --lines 200</code>	Últimas 200 líneas
<code>pm2 restart olea-server</code>	Reinicio tras un despliegue
<code>pm2 monit</code>	Monitor interactivo de CPU y memoria
<code>pm2 flush</code>	Vacía los logs (hazlo cada mes)

9.3 Rotación de logs

Sin rotación, `salida.log` crece hasta llenar el disco. Además tu `adaptHandler` escribe en `api-errors.log` con `appendFileSync` y ese archivo **nunca se trunca solo**:

```
pm2 install pm2-logrotate
pm2 set pm2-logrotate:max_size 20M
pm2 set pm2-logrotate:retain 14
pm2 set pm2-logrotate:compress true
```

Para `api-errors.log`, agrega una tarea mensual que lo recorte:

```
$log = "C:\olea\app\api-errors.log"
if ((Get-Item $log -EA SilentlyContinue).Length -gt 50MB) {
    Move-Item $log "C:\olea\logs\api-errors-$(Get-Date -f yyyyMM).log" -Force
}
```

10. Fase 7 — RespalDOS automáticos a R2

Tiempo estimado: 30 minutos. **El punto más importante de todo el documento.**

Lo que estás asumiendo al migrar

En Prisma Postgres los respaldos los hacía el proveedor. En tu PC,

si no configuras esto y el disco muere, pierdes todo

: OTs, empleados, nómina, cotizaciones, proyectos. No hay recuperación posible. Un respaldo local no basta: si se quema la PC, se quema el respaldo. Por eso van a R2.

10.1 Script de respaldo

Crea `C:\olea\respaldo.ps1`. Usa `rclone` (gratis) para subir a R2 con el mismo bucket que ya tienes:

```
# C:\olea\respaldo.ps1
$ErrorActionPreference = "Stop"
$fecha = Get-Date -Format "yyyy-MM-dd_HH:mm"
$archivo = "C:\olea\backups\olea-$fecha.dump"
$env:PGPASSWORD = "CONTRASENA_DE_olea_app"

# 1) Volcado comprimido en formato custom (-Fc): el más rápido de restaurar
pg_dump -U olea_app -h localhost -d olea -Fc -f $archivo
if (-not (Test-Path $archivo)) { throw "El pg_dump no generó archivo" }

$mb = [math]::Round((Get-Item $archivo).Length / 1MB, 2)
Write-Host "Respaldo generado: $archivo ($mb MB)"

# 2) Subir a Cloudflare R2 (remoto 'r2' configurado con rclone config)
rclone copy $archivo r2:TU-BUCKET/backups/ --s3-no-check-bucket

# 3) Conservar 14 días en local (en R2 conserva más, cuesta centavos)
Get-ChildItem C:\olea\backups\olea-*.dump |
    Where-Object { $_.LastWriteTime -lt (Get-Date).AddDays(-14) } |
    Remove-Item -Force

# 4) Aviso por Telegram — reutiliza el bot que ya tienes
$token = "TU_TELEGRAM_BOT_TOKEN"; $chat = "TU_CHAT_ID"
$texto = "Respaldo Olea OK — $fecha — $mb MB"
Invoke-RestMethod -Method Post -Uri "https://api.telegram.org/bot$token/sendMessage" `
    -Body @{ chat_id = $chat; text = $texto } | Out-Null
```

10.2 Configurar rclone contra R2

```
winget install Rclone.Rclone
rclone config
# n) nuevo remoto -> nombre: r2
# storage: s3 -> provider: Cloudflare
# access_key_id / secret_access_key: los mismos R2_ACCESS_KEY / R2_SECRET_KEY del .env
# endpoint: https://XXXXX.r2.cloudflarestorage.com
# region: auto

# Probar
rclone ls r2:
```

10.3 Programar la tarea

```
# PowerShell como administrador – respaldo diario a las 2:00 a.m.
$accion = New-ScheduledTaskAction -Execute "powershell.exe" `
    -Argument "-ExecutionPolicy Bypass -File C:\olea\respaldo.ps1"
$disparo = New-ScheduledTaskTrigger -Daily -At 2:00AM
$config = New-ScheduledTaskSettingsSet -StartWhenAvailable `
    -DontStopIfGoingOnBatteries -RunOnlyIfNetworkAvailable

Register-ScheduledTask -TaskName "Olea-Respaldo-Diario" `
    -Action $accion -Trigger $disparo -Settings $config `
    -User "SYSTEM" -RunLevel Highest

# Ejecutarla ahora mismo para verificar que funciona
Start-ScheduledTask -TaskName "Olea-Respaldo-Diario"
```

10.4 Ensayo de restauración

Un respaldo que nunca probaste no es un respaldo. **Una vez al mes**, restaura en una base de prueba:

```
psql -U postgres -c "CREATE DATABASE olea_prueba;"
pg_restore -U postgres -d olea_prueba --no-owner C:\olea\backups\olea-FECHA.dump
psql -U postgres -d olea_prueba -c "SELECT count(*) FROM \"WorkOrder\";"
psql -U postgres -c "DROP DATABASE olea_prueba;"
```

10.5 Qué más respaldar

Elemento	Dónde vive	Estrategia
Base de datos	PostgreSQL local	pg_dump diario a R2 (arriba)
Archivos, fotos, PDFs	Cloudflare R2	Ya están en la nube. Activa versionado del bucket en el panel de R2
Código	GitHub	Ya respaldado. Haz git push de todo cambio local
.env	Solo en la PC	Cópialo a un gestor de contraseñas. No está en Git (y así debe seguir)
Config de cloudflared	.cloudflared\	Copia el JSON de credenciales a lugar seguro

11. Fase 8 — Seguridad

Al salir de Vercel pierdes su capa de protección por defecto. Estas medidas la reponen.

11.1 Firewall de Windows

```
# PostgreSQL SOLO accesible desde la propia PC – jamás desde la red
New-NetFirewallRule -DisplayName "Bloquear Postgres externo" -Direction Inbound `
  -LocalPort 5432 -Protocol TCP -Action Block

# El puerto 3001 tampoco se expone: el túnel se conecta desde dentro (saliente)
New-NetFirewallRule -DisplayName "Bloquear Node externo" -Direction Inbound `
  -LocalPort 3001 -Protocol TCP -Action Block

# Excepción: si quieres acceso desde la red local de la oficina
New-NetFirewallRule -DisplayName "Node LAN" -Direction Inbound -LocalPort 3001 `
  -Protocol TCP -RemoteAddress 192.168.1.0/24 -Action Allow
```

Ventaja del túnel

Con Cloudflare Tunnel o Tailscale

no abres un solo puerto entrante

. La PC inicia una conexión saliente y el tráfico viaja por ahí. Un escaneo desde internet a tu IP no encuentra nada abierto. Es más seguro que un servidor con puertos publicados.

11.2 CORS cerrado

Hoy `server.js` usa `app.use(cors())` sin parámetros: **acepta peticiones de cualquier origen**. En desarrollo no importa; expuesto a internet, sí. Aplica la configuración restrictiva de la sección 6.2.

11.3 Límite de peticiones (rate limit)

El endpoint de login es el blanco obvio de fuerza bruta. Sin Vercel enfrente, protégelo tú:

```
npm install express-rate-limit
```

```
// en server.js, antes de las rutas
import ratelimit from 'express-rate-limit';

const limiteLogin = ratelimit({
  windowMs: 15 * 60 * 1000,      // 15 minutos
  max: 10,                      // 10 intentos por IP
  message: { error: 'Demasiados intentos. Espera 15 minutos.' },
  standardHeaders: true,
});

const limiteGeneral = ratelimit({ windowMs: 60 * 1000, max: 300 });

app.use('/api', limiteGeneral);
app.post('/api/login', limiteLogin, adaptHandler(loginHandler)); // sustituye la línea actual

// Necesario detrás del túnel para que vea la IP real y no la del proxy
app.set('trust proxy', 1);
```

11.4 Cabeceras de seguridad

```
npm install helmet
```

```
import helmet from 'helmet';
app.use(helmet({
  contentSecurityPolicy: false, // desactivada: tu PWA usa inline styles y Leaflet
  crossOriginEmbedderPolicy: false,
}));
```

11.5 Reducir el tamaño de cuerpo aceptado

`server.js` acepta `json({ limit: '50mb' })`. Con la subida directa a R2 que ya implementaste (commit `c060119`), los archivos grandes **ya no pasan por la API**. Bájalo a `10mb`: menos superficie para un ataque de agotamiento de memoria.

11.6 Higiene general

- **JWT_SECRET**: mínimo 32 caracteres aleatorios. Génalo con `node -e "console.log(require('crypto').randomBytes(48).toString('base64'))"`. Si lo cambias, todas las sesiones activas se invalidan (avisa antes).
- **Usuario de BD**: `olea_app` nunca debe ser superusuario. Ya lo creamos así en 5.1.
- **Antivirus**: agrega `C:\olea` y el directorio de datos de PostgreSQL a las exclusiones de Defender. El escaneo en tiempo real sobre los archivos de la base degrada mucho el rendimiento.
- **Actualizaciones**: `npm audit` una vez al mes; parches de Windows en la ventana de mantenimiento.
- **Cloudflare Access** (solo Opción 1, gratis hasta 50 usuarios): puedes exigir correo corporativo para llegar a rutas administrativas. Una segunda puerta antes del login de la app.

11.7 Rutas públicas — revisión

Tu `server.js` expone sin token: `/api/appointments` (GET/POST), `/api/warranty` (GET/POST) y `/api/portal`. Eso es correcto por diseño (portal de cliente y citas de tienda), pero **ahora están directamente en internet sin el filtro de Vercel**. Verifica que:

- Validen y saneen todo lo que reciben antes de escribir en la BD.
- No devuelvan más campos de los necesarios (nada de datos internos de empleados o precios de costo).
- Estén cubiertos por `limiteGeneral` del rate limit.

12. Fase 9 — Contingencia y vuelta atrás

12.1 Mantén Vercel vivo como respaldo

Cuesta \$0 y es tu seguro. La configuración de respaldo:

- El proyecto de Vercel sigue conectado al mismo repositorio de GitHub y desplegando `main`.
- En las variables de entorno de Vercel, `DATABASE_URL` apunta a **Prisma Postgres** (no a tu PC: es inalcanzable desde ahí).
- Las llamadas a `emitir()` que agregaste devuelven `false` allá y no rompen nada; el sistema funciona sin realtime, como hoy.
- Si tu BD principal ya es la local, el respaldo de Vercel tendrá datos antiguos: sirve para **consultar** y para operación degradada, no para capturar en paralelo.

12.2 Procedimiento de cambio de emergencia

Situación	Acción inmediata	Tiempo
Se fue la luz	El UPS aguanta 10–20 min. Si el corte se extiende, apaga limpio con <code>pm2 stop all</code> y <code>pg_ctl stop</code>	—
Se cayó el internet	Comparte datos móviles a la PC; el túnel se reconecta solo	5 min
La PC no arranca	Cambia el DNS de <code>olea.tudominio.com</code> a un CNAME hacia Vercel. Sistema en modo lectura con datos del último respaldo	15 min
Corrupción de BD	<code>pg_restore</code> del dump de R2 más reciente	20–40 min
El despliegue rompió algo	<code>git reset --hard HEAD~1 && npm run build && pm2 restart olea-server</code>	5 min

12.3 Monitoreo gratuito

Necesitas enterarte de que el sistema se cayó antes que tus usuarios:

- **UptimeRobot** (gratis, 50 monitores): vigila `https://olea.tudominio.com/api/health` cada 5 minutos y te avisa por correo y Telegram.
- **Script local de vigilancia**: una tarea programada cada 10 minutos que consulta `/api/health` y, si falla, reinicia PM2 y te manda un mensaje por el bot de Telegram que ya tienes.

```
# C:\olea\vigilancia.ps1
try {
    $r = Invoke-RestMethod http://localhost:3001/api/health -TimeoutSec 10
    if (-not $r.ok) { throw "health devolvió ok=false" }
} catch {
    pm2 restart olea-server
    $token="TU_TOKEN"; $chat="TU_CHAT"
    Invoke-RestMethod -Method Post -Uri "https://api.telegram.org/bot$token/sendMessage" `
        -Body @{ chat_id=$chat; text="ALERTA: servidor Olea reiniciado. $($_.Exception.Message)" }
}
```

12.4 Costos reales del esquema final

Concepto	Costo mensual	Nota
Node, PostgreSQL, PM2, rclone	\$0	Software libre
Cloudflare Tunnel + DNS + WAF	\$0	Plan gratuito, ancho de banda ilimitado
Dominio .com	~\$17 MXN	~\$200 MXN al año (única partida pagada)
Cloudflare R2	\$0	10 GB gratis; sin cargos por salida de datos
Vercel (respaldo)	\$0	Plan Hobby
Prisma Postgres (respaldo)	\$0	Plan gratuito
UptimeRobot	\$0	Plan gratuito
Electricidad de la PC 24/7	~\$60–120 MXN	Estimado a 50–80 W continuos
TOTAL	~\$80–140 MXN/mes	Con Tailscale en vez de dominio propio: ~\$60–120 MXN

13. Anexos

13.1 Checklist de puesta en marcha

✓	Tarea	Fase
☐	PC configurada sin suspensión, con IP fija y UPS conectado	0
☐	Node, Git, PostgreSQL y cloudflared instalados y verificados	1
☐	Repositorio clonado en <code>C:\olea\app</code> (fuera de OneDrive)	0
☐	<code>.env</code> creado con R2, JWT y DATABASE_URL correctos	1
☐	Base <code>olea</code> y usuario <code>olea_app</code> creados	2
☐	Esquema aplicado con <code>prisma migrate deploy</code>	2
☐	Datos migrados y conteos verificados contra la nube	2
☐	Ajuste de SSL condicional en <code>prisma.js</code>	2
☐	<code>server.js</code> sirve <code>dist/</code> con fallback de SPA por regex	3
☐	<code>npm run build</code> exitoso y app accesible en la LAN	3
☐	Túnel activo con HTTPS y URL pública funcionando	4
☐	PWA instalable desde el celular de un técnico (prueba real)	4
☐	Socket.IO conectando: <code>/api/health</code> reporta usuarios	5
☐	Al menos un evento en vivo probado entre dos dispositivos	5
☐	PM2 configurado, <code>pm2 save</code> hecho y sobrevive a un reinicio	6
☐	Respaldo diario corriendo y subiendo a R2	7
☐	Restauración de prueba ejecutada con éxito	7
☐	CORS cerrado, rate limit y helmet activos	8
☐	Firewall bloqueando 5432 y 3001 desde el exterior	8
☐	UptimeRobot vigilando <code>/api/health</code>	9
☐	Vercel sigue desplegando como respaldo	9
☐	<code>.env</code> guardado en el gestor de contraseñas	7

13.2 Comandos de referencia rápida

```
### SERVIDOR ###
pm2 list                # estado
pm2 restart olea-server # reiniciar
pm2 logs olea-server --lines 100 # ver logs

### DESPLIEGUE ###
C:\olea\actualizar.ps1 # pull + build + restart con respaldo previo

### BASE DE DATOS ###
psql -U olea_app -d olea # consola SQL
npx prisma studio        # explorador visual en el navegador
npx prisma migrate deploy # aplicar migraciones pendientes
pg_dump -U olea_app -d olea -Fc -f respaldo.dump # respaldo manual
pg_restore -U olea_app -d olea --clean respaldo.dump

### TÚNEL ###
cloudflared tunnel list
cloudflared tunnel info olea-server
Get-Service cloudflared

### DIAGNÓSTICO ###
curl http://localhost:3001/api/health
netstat -ano | findstr :3001 # quién ocupa el puerto
Get-Content C:\olea\logs\error.log -Tail 50 -Wait
```

13.3 Solución de problemas

Síntoma	Causa probable	Solución
<code>PathError: Missing parameter name</code> al arrancar	Usaste <code>app.get('*')</code> en Express 5	Cambia al patrón regex de la sección 6.2
<code>The server does not support SSL connections</code>	Postgres local sin SSL y el pool lo exige	Aplica el ajuste condicional de la sección 5.5
El frontend carga pero <code>/api</code> da 404	El <code>express.static</code> quedó antes de las rutas de API	Mueve los estáticos al final del archivo
Socket.IO conecta y se desconecta en bucle	CORS del socket sin el origen, o token vencido	Revisa <code>ORIGENES</code> en <code>initRealtime</code> y renueva sesión
El socket cae cada ~100 segundos	Timeout del túnel	Agrega <code>keepAliveTimeout: 90s</code> al <code>config.yml</code>
<code>too many clients already</code>	<code>DB_POOL_MAX</code> mayor que <code>max_connections</code> de Postgres	Baja el pool o sube <code>max_connections</code> en <code>postgresql.conf</code>
La PWA no se instala en el celular	Estás sirviendo por HTTP, no HTTPS	Completa la Fase 4; no hay atajo
Sesiones cerrándose solas	<code>JWT_SECRET</code> distinto entre la PC y Vercel	Usa exactamente el mismo valor en ambos
Fotos que no suben	CORS del bucket R2 sin el dominio nuevo	Agrega el dominio en <code>docs/r2-cors.json</code> y aplícalo en el panel de R2
Todo lento después de semanas	Postgres sin mantenimiento	<code>VACUUM ANALYZE;</code> y revisa índices con <code>pg_stat_user_tables</code>

13.4 Orden de trabajo sugerido

Día	Actividad	Resultado
1	Fases 0, 1 y 3 + Fase 4 con Tailscale	Ruta A funcionando: API en la PC, frontend en Vercel. Reversible al instante.
2	Fase 5 completa	Tiempo real en OTs y GPS. Ya justificaste la migración.
3	Fases 6 y 7	Servicio automático y respaldos. Ya es operable sin ti.
4	Fase 2 (migración de BD) en fin de semana	Ruta B: independencia total de Prisma.
5	Fases 8 y 9 + dominio propio y Cloudflare Tunnel	Producción endurecida con plan de contingencia.

Una última cosa

No migres todo en un solo día. Cada fase de este documento deja el sistema en un estado funcional y reversible. Si algo sale mal en la fase 4, sigues teniendo Vercel; si algo sale mal en la fase 2, sigues teniendo Prisma Postgres con los datos intactos. Ese es el diseño: nunca quedarte sin sistema mientras aprendes a operar el tuyo.